

Отчёт по лабораторной работе 08

Простейший вариант

Симонова Виктория Игоревна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	9
	Список литературы	65

Список иллюстраций

4.1	Создание нового релиза v.8.0.0	9
4.2	Запуск Julia	10
4.3	Установка пакетов Julia	10
4.4	Установка пакетов	11
4.5	Установка пакетов	11
4.6	Установка пакетов	11
4.7	Запуск скрипта scripts/sir_model.jl	11
4.8	jupyter-notebook от scripts/sir_model.jl	12
4.9	Запуск скрипта scripts/task_8_5_1.jl	13
4.10	jupyter-notebook от scripts/ross_model.jl	14
4.11	Запуск скрипта scripts/task_8_5_2.jl	14
4.12	jupyter-notebook от scripts/task_8_5_2.jl	15
4.13	Запуск скрипта scripts/task_8_5_3.j	15
4.14	jupyter-notebook от scripts/task_8_5_3.j	16
4.15	Запуск скрипта scripts/task_8_5_4.j	17
4.16	jupyter-notebook от scripts/task_8_5_4.j	17
4.17	Запуск скрипта scripts/task_8_5_5.j	18
4.18	jupyter-notebook от scripts/task_8_5_5.j	18
4.19	Запуск скрипта scripts/task_8_5_6.jl	19
4.20	график скрипта scripts/task_8_5_6.jl	19
4.21	jupyter-notebook от scripts/task_8_5_6.jl	20
4.22	Запуск скрипта scripts/task_8_5_7.j	21
4.23	jupyter-notebook от scripts/task_8_5_7.j	22
4.24	Настройка _quarto.yml	25
4.25	Подключение файлов	26

Список таблиц

1 Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель для углублённого исследования эпидемиологических процессов.

2 Задание

2.1 Базовые задачи Создание рабочего каталога для кода и установка необходимых пакетов (ResumableFunctions, ConcurrentSim, Distributions, DataFrames, StatsPlots, BenchmarkTools, CSV).

Реализация базовой дискретно-событийной SIR модели на языке Julia.

Визуализация результатов моделирования.

2.2 Дополнительные задания (п. 8.5) Анализ чувствительности к параметрам – проведение нескольких прогонов с разными значениями β , σ , γ , анализ ключевых метрик (пик I, время пика, итоговая доля переболевших).

Детерминированная длительность болезни – замена экспоненциального времени выздоровления на фиксированную величину, сравнение со стохастической версией.

Оценка производительности – измерение времени выполнения для различных размеров популяции с помощью макроса **benchmark**.

Сохранение результатов в CSV – автоматическое сохранение таблицы результатов с уникальным именем, содержащим параметры запуска.

Добавление демографических событий – расширение модели с добавлением смертности (с интенсивностью μ) и рождаемости (с интенсивностью ν).

Вакцинация – реализация стратегии вакцинации в заданный момент времени или при превышении порога инфицированных.

Модель SEIR – введение латентного периода (статус :E) с экспоненциальным временем перехода в инфекционное состояние.

3 Теоретическое введение

3.1 Дискретно-событийное моделирование Дискретно-событийное моделирование (DES) — метод имитационного моделирования, при котором состояние системы изменяется только в дискретные моменты времени, соответствующие наступлению событий. Виртуальное время «перескакивает» от одного события к другому, что обеспечивает высокую эффективность по сравнению с непрерывным моделированием.

3.2 SIR модель SIR — классическая эпидемиологическая модель, разделяющая популяцию на три группы:

S (Susceptible) – восприимчивые,

I (Infected) – инфицированные,

R (Recovered) – выздоровевшие.

3.3 Стохастическая DES реализация В дискретно-событийной стохастической версии каждый индивид моделируется агентом. Время до следующего контакта распределено экспоненциально с параметром λ , время до выздоровления — экспоненциально с параметром γ . При контакте восприимчивого с инфицированным заражение происходит с вероятностью β .

3.4 Расширения модели SEIR – добавляет состояние E (Exposed, латентный период) с интенсивностью перехода σ .

Демография – добавляет рождаемость (ν) и смертность (μ) для исследования эндемического равновесия.

Вакцинация – переводит часть восприимчивых в состояние R.

3.5 Используемые библиотеки Julia Библиотека Назначение ConcurrentSim

Дискретно-событийное ядро ResumableFunctions Возобновляемые функции
Distributions Генерация случайных величин DataFrames, CSV Работа с данны-
ми StatsPlots Визуализация BenchmarkTools Оценка производительности

4 Выполнение лабораторной работы

4.0.1 1. Создание нового релиза

Создаем новый релиз v.8.0.0

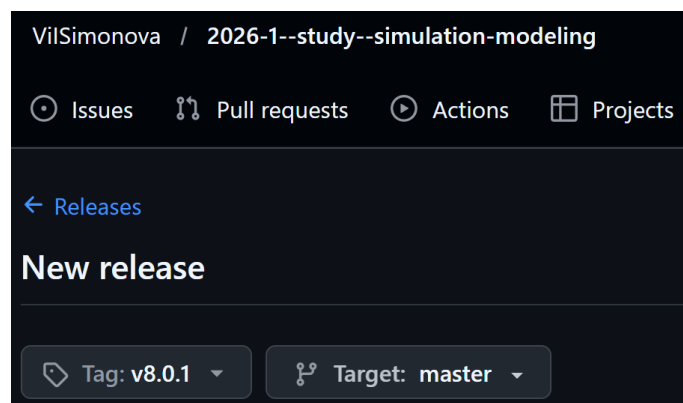


Рисунок 4.1: Создание нового релиза v.8.0.0

4.0.2 2. Настройка окружения Julia

Запускаем Julia и установку пакетов


```
t=. -e 'using Pkg; Pkg.add("Literate")'
Resolving package versions...
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/Project.toml`
[98b081ad] + Literate v2.21.0
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/Manifest.toml`
[b5f81e59] + IOCapture v1.0.0
[98b081ad] + Literate v2.21.0
Precompiling project...
1 dependency successfully precompiled in 4 seconds. 251 already precompiled.
```

Рисунок 4.4: Установка пакетов

```
t=. -e 'using Pkg; Pkg.add("CSV")'
Resolving package versions...
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/Project.toml`
[336ed68f] + CSV v0.10.16
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/Manifest.toml`
[336ed68f] + CSV v0.10.16
[48062228] + FilePathsBase v0.9.24
[ea10d353] + WeakRefStrings v1.4.3
[76ecccc3] + WorkerUtilities v1.6.1
```

Рисунок 4.5: Установка пакетов

```
t=. -e 'using Pkg; Pkg.add("StatsPlots"); Pkg.add("DataFrames");'
Resolving package versions...
No Changes to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/Project.toml`
No Changes to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/Manifest.toml`
Resolving package versions...
No Changes to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/Project.toml`
No Changes to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/Manifest.toml`
```

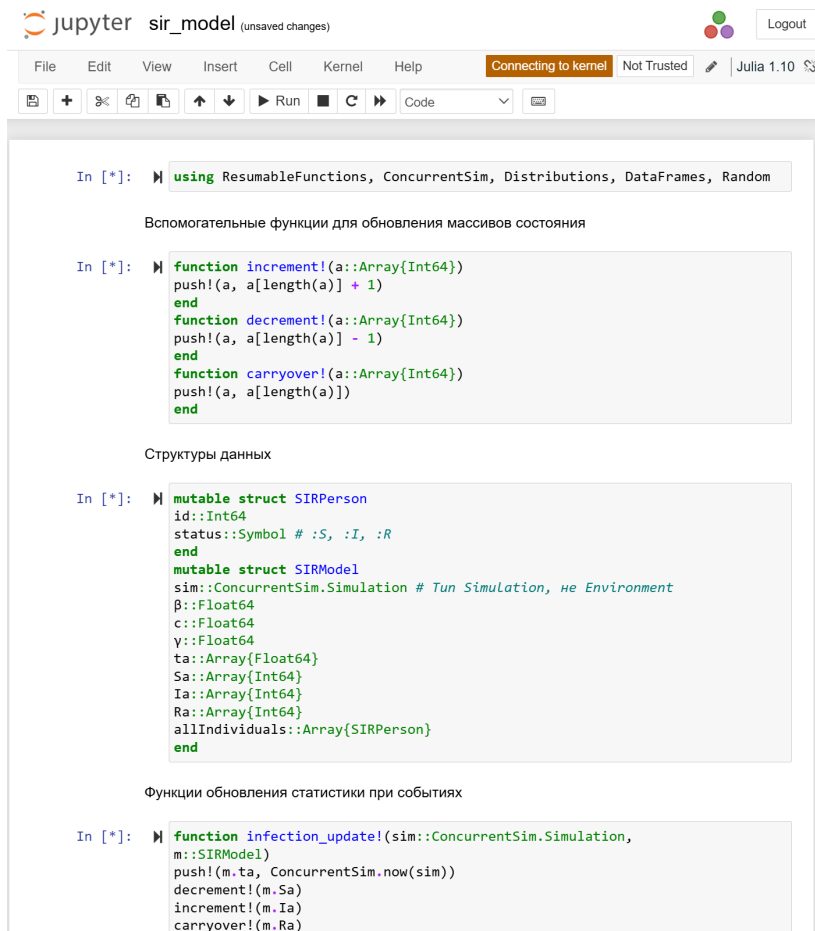
Рисунок 4.6: Установка пакетов

Запуск первого скрипта scripts/sir_model.jl

```
t=. scripts/sir_des.jl
=== ОЦЕНКА ПРОИЗВОДИТЕЛЬНОСТИ ===

N=100: время = 0.05 мс, память = 0.0 МБ
N=500: время = 0.04 мс, память = 0.0 МБ
N=1000: время = 0.04 мс, память = 0.0 МБ
N=2000: время = 0.03 мс, память = 0.0 МБ
N=5000: время = 0.04 мс, память = 0.0 МБ
```

Рисунок 4.7: Запуск скрипта scripts/sir_model.jl



The screenshot shows a Jupyter Notebook interface with the title 'sir_model' and a status '(unsaved changes)'. The top bar includes a menu (File, Edit, View, Insert, Cell, Kernel, Help), a 'Connecting to kernel' button, a 'Not Trusted' warning, and a 'Logout' button. Below the menu is a toolbar with icons for saving, adding cells, undo, redo, running, and code editing. The notebook content is organized into sections with headings in Russian:

- Вспомогательные функции для обновления массивов состояния**: A code cell containing three functions: `increment!`, `decrement!`, and `carryover!`, each taking an array of `Int64` and pushing or popping an element.
- Структуры данных**: A code cell defining two mutable structs: `SIRPerson` (with fields `id`, `status`) and `SIRModel` (with fields `sim`, `β`, `c`, `γ`, `ta`, `Sa`, `Ia`, `Ra`, and `allIndividuals`).
- Функции обновления статистики при событиях**: A code cell defining the `infection_update!` function, which updates the simulation state by pushing a new time point, decrementing `Sa`, incrementing `Ia`, and carrying over `Ra`.

Рисунок 4.8: jupyter-notebook от scripts/sir_model.jl

Создание, запуск следующего скрипта scripts/task_8_5_1.jl и создание его производных форматов

```

t=. scripts/task_8_5_1.jl
Запуск:  $\beta=0.03$ ,  $c=5.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.03$ ,  $c=5.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.03$ ,  $c=5.0$ ,  $\gamma=0.3$ 
Запуск:  $\beta=0.03$ ,  $c=10.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.03$ ,  $c=10.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.03$ ,  $c=10.0$ ,  $\gamma=0.3$ 
Запуск:  $\beta=0.03$ ,  $c=15.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.03$ ,  $c=15.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.03$ ,  $c=15.0$ ,  $\gamma=0.3$ 
Запуск:  $\beta=0.05$ ,  $c=5.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.05$ ,  $c=5.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.05$ ,  $c=5.0$ ,  $\gamma=0.3$ 
Запуск:  $\beta=0.05$ ,  $c=10.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.05$ ,  $c=10.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.05$ ,  $c=10.0$ ,  $\gamma=0.3$ 
Запуск:  $\beta=0.05$ ,  $c=15.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.05$ ,  $c=15.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.05$ ,  $c=15.0$ ,  $\gamma=0.3$ 
Запуск:  $\beta=0.07$ ,  $c=5.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.07$ ,  $c=5.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.07$ ,  $c=5.0$ ,  $\gamma=0.3$ 
Запуск:  $\beta=0.07$ ,  $c=10.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.07$ ,  $c=10.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.07$ ,  $c=10.0$ ,  $\gamma=0.3$ 
Запуск:  $\beta=0.07$ ,  $c=15.0$ ,  $\gamma=0.2$ 
Запуск:  $\beta=0.07$ ,  $c=15.0$ ,  $\gamma=0.25$ 
Запуск:  $\beta=0.07$ ,  $c=15.0$ ,  $\gamma=0.3$ 

=== РЕЗУЛЬТАТЫ АНАЛИЗА ЧУВСТВИТЕЛЬНОСТИ ===
27x7 DataFrame
Row |  $\beta$       c       $\gamma$       peak_I  peak_time  final_R  total_infe
cted |
    | Float64  Float64  Float64  Int64    Float64    Int64    Int64
-----|-----
1 | 0.03     5.0     0.2      13    30.0504      52
52
2 | 0.03     5.0     0.25     11    0.221702     24

```

Рисунок 4.9: Запуск скрипта scripts/task_8_5_1.jl

```

In [*]: using Random, DataFrames, CSV, StatsPlots
include("../src/sir_model.jl")

function sensitivity_analysis()
    u0 = [990, 10, 0]
    tmax = 40.0

    Вaryируемые параметры

In [*]: betas = [0.03, 0.05, 0.07]
cs = [5.0, 10.0, 15.0]
gammas = [0.2, 0.25, 0.3]

results = DataFrame{
    β=Float64[], c=Float64[], γ=Float64[],
    peak_I=Int64[], peak_time=Float64[], final_R=Int64[],
    total_infected=Int64[]
}

for β in betas
    for c in cs
        for γ in gammas
            println("Заныск: β=$β, c=$c, γ=$γ")

            Random.seed!(1234) # Фиксируем seed для воспроизводимости
            p = [β, c, γ]
            m = MakeSIRModel(u0, p)
            activate(m)
            sir_run(m, tmax)
            data = out(m)

            peak_I = maximum(data.I)
            peak_time = data.t[argmax(data.I)]
            final_R = last(data.R)
            total_infected = final_R - u0[3] # Сколько заболело всего

            push!(results, (β, c, γ, peak_I, peak_time, final_R, total_infected))
        end
    end
end

```

Рисунок 4.10: jupyter-notebook от scripts/ross_model.jl

Создание, запуск следующего скрипта scripts/task_8_5_2.jl и создание его производных форматов

```

t=. scripts/task_8_5_2.jl
График сохранён в plots/comparison_stoch_det.png

```

Рисунок 4.11: Запуск скрипта scripts/task_8_5_2.jl

```

In [ ]: using Random, StatsPlots, DataFrames
        include("../src/sir_model.jl")

Модифицированная версия live с фиксированным временем болезни

In [ ]: @resumable function live_deterministic(env::ConcurrentSim.Simulation,
                                             individual::SIRPerson,
                                             m::SIRModel)
        while individual.status == :S
            @yield timeout(env, rand(Exponential(1/m.c)))
        end

        Выбор случайного собеседника

In [ ]: alter = individual
        while alter == individual
            N = length(m.allIndividuals)
            index = rand(DiscreteUniform(1, N))
            alter = m.allIndividuals[index]
        end

        if alter.status == :I
            if rand() < m.β
                individual.status = :I
                infection_update!(env, m)
            end
        end

        if individual.status == :I

            ФИКСИРОВАННОЕ время болезни (вместо экспоненциального)

In [ ]: @yield timeout(env, 1/m.γ)
        individual.status = :R
        recovery_update!(env, m)
        end
    end

    Переопределяем функцию activate для использования детерминированной версии
  
```

Рисунок 4.12: jupyter-notebook от scripts/task_8_5_2.jl

Создание, запуск следующего скрипта scripts/task_8_5_3.jl и создание его производных форматов

```

t=. scripts/task_8_5_3.jl
=== ОЦЕНКА ПРОИЗВОДИТЕЛЬНОСТИ ===

N=100: время = 0.05 мс, память = 0.0 МБ
N=500: время = 0.04 мс, память = 0.0 МБ
N=1000: время = 0.04 мс, память = 0.0 МБ
N=2000: время = 0.04 мс, память = 0.0 МБ
N=5000: время = 0.04 мс, память = 0.0 МБ
  
```

Рисунок 4.13: Запуск скрипта scripts/task_8_5_3.j

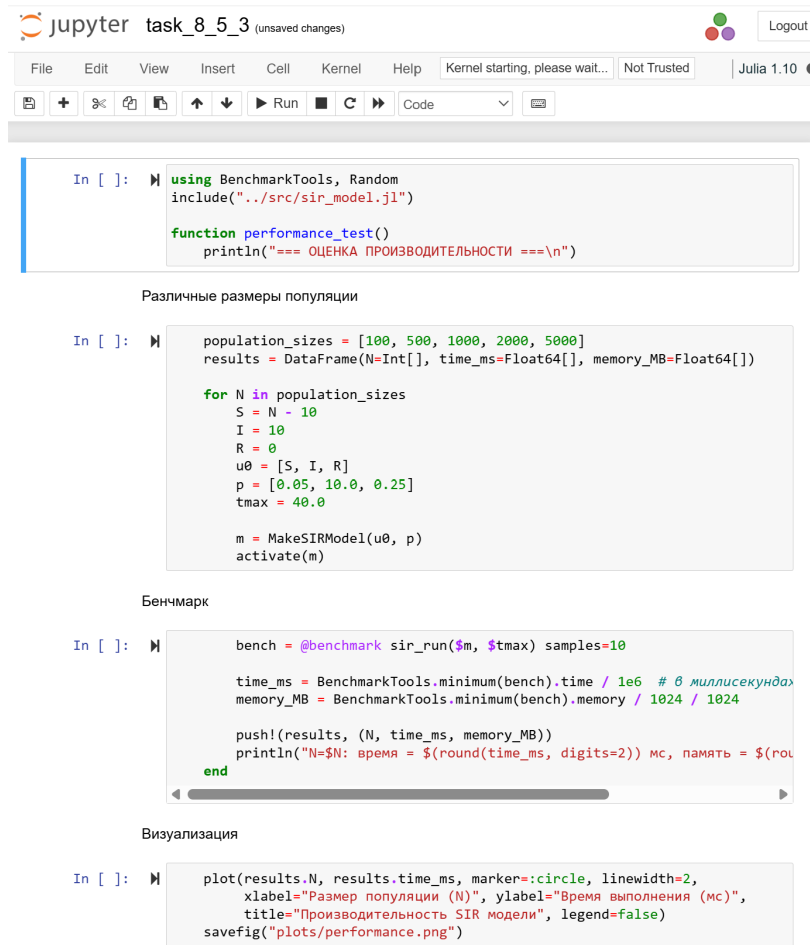


Рисунок 4.14: jupyter-notebook от scripts/task_8_5_3.j

Создание, запуск следующего скрипта scripts/task_8_5_4.j и создание его производных форматов


```
t=. scripts/task_8_5_4.jl
✓ Результаты сохранены:
Данные: data/sims/sir_S=990_I=10_R=0_β=0.05_c=10.0_γ=0.25_tmax=40.0_20260524_105459.csv
Метаданные: data/sims/sir_S=990_I=10_R=0_β=0.05_c=10.0_γ=0.25_tmax=40.0_20260524_105459_metadata.csv

Первые 5 строк сохранённых данных:
5x4 DataFrame

```

Row	t Float64	S Int64	I Int64	R Int64
1	0.0	990	10	0
2	0.110851	989	11	0
3	0.249585	988	12	0
4	0.314392	987	13	0
5	0.418738	987	12	1

Рисунок 4.15: Запуск скрипта scripts/task_8_5_4.j

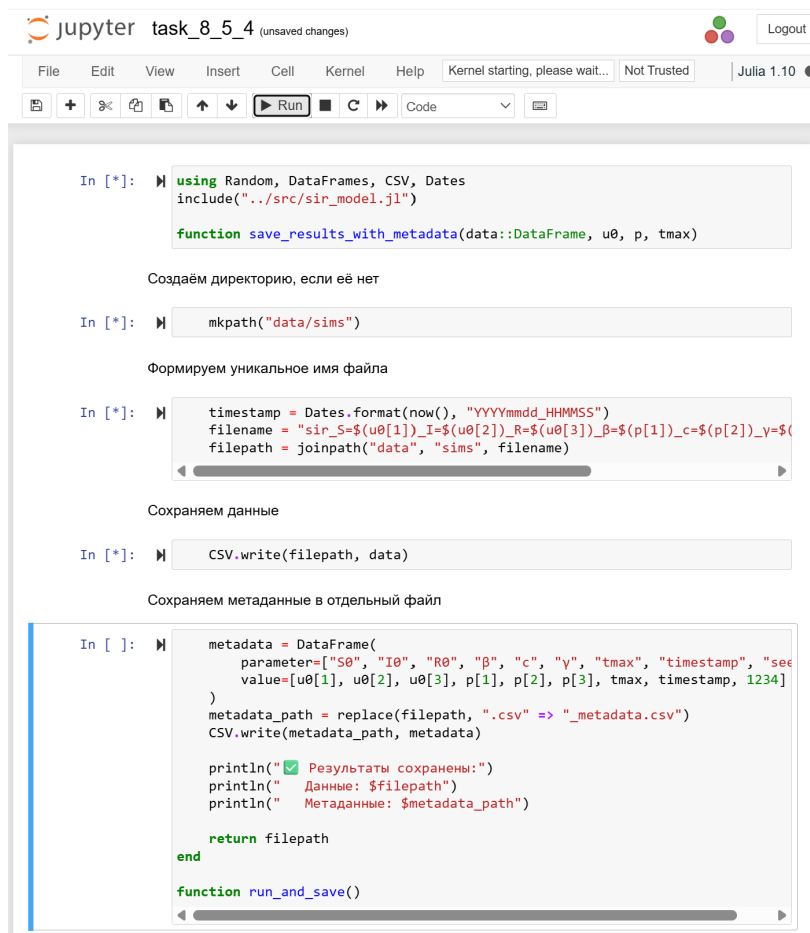


Рисунок 4.16: jupyter-notebook от scripts/task_8_5_4.j

Создание, запуск следующего скрипта scripts/task_8_5_5.j и создание его

производных форматов

```
t=. scripts/task_8_5_5.jl
=== МОДЕЛЬ SIR С ДЕМОГРАФИЕЙ ===
Параметры:
- Смертность ( $\mu$ ): 0.01 (средняя жизнь = 100.0 ед. времени)
- Рождаемость ( $\nu$ ): 0.012 (средний интервал = 83.333333333333 ед. вре
мени)
- Длительность симуляции: 100.0
```

Рисунок 4.17: Запуск скрипта scripts/task_8_5_5.j

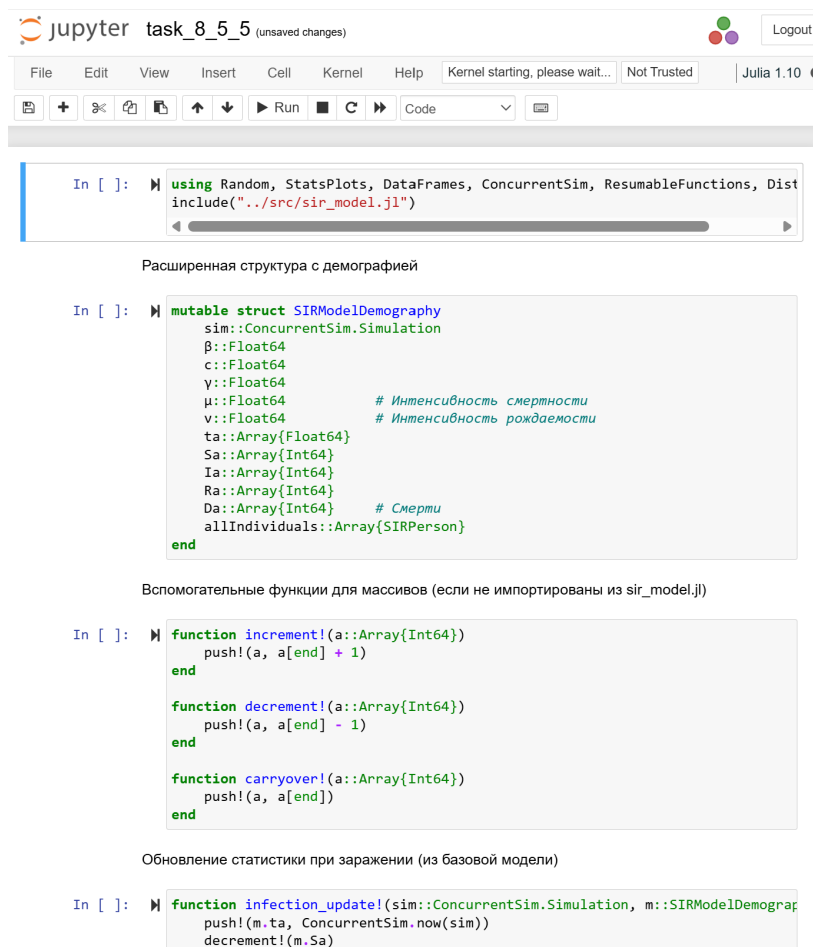


Рисунок 4.18: jupyter-notebook от scripts/task_8_5_5.j

Создание, запуск следующего скрипта scripts/task_8_5_6.jl и создание его производных форматов

```
t=. scripts/task_8_5_6.jl
Запуск с долей вакцинации: 0.0%
Запуск с долей вакцинации: 20.0%
Вакцинировано 174 человек в момент времени 10.0
```

Рисунок 4.19: Запуск скрипта scripts/task_8_5_6.jl

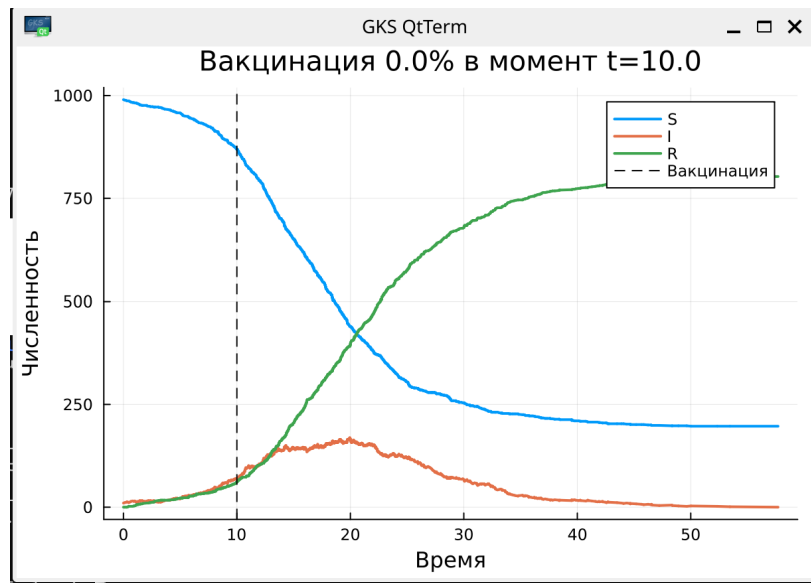


Рисунок 4.20: график скрипта scripts/task_8_5_6.jl


```

t=. scripts/task_8_5_7.jl
=== Запуск SIR модели ===
=== Запуск SEIR модели ===

=== РЕЗУЛЬТАТЫ СРАВНЕНИЯ ===
SIR модель:
- Пик инфекции: 169 в момент t = 19.949690024371733
- Всего переболело: 803

SEIR модель (с латентным периодом 1/σ = 2.0 дней):
- Пик инфекции (I): 174 в момент t = 23.827183634356
- Пик латентных (E): 89 в момент t = 19.195632837013736
- Всего переболело: 870

=== АНАЛИЗ ЗАДЕРЖКИ ===
Пик инфекции в SIR: t = 19.95
Пик инфекции в SEIR: t = 23.83
Задержка пика из-за латентного периода: 3.88 ед. времени

=== ИССЛЕДОВАНИЕ ВЛИЯНИЯ ЛАТЕНТНОГО ПЕРИОДА ===
Запуск с σ = 0.2 (латентный период = 5.0 дней)
Запуск с σ = 0.5 (латентный период = 2.0 дней)
Запуск с σ = 1.0 (латентный период = 1.0 дней)
Запуск с σ = 2.0 (латентный период = 0.5 дней)

Результаты исследования:
4x4 DataFrame

```

Row	σ	latent_days	peak_I	peak_time
	Float64	Float64	Int64	Float64
1	0.2	5.0	126	37.8532
2	0.5	2.0	174	23.8272
3	1.0	1.0	152	23.0873
4	2.0	0.5	205	15.475

```

✓ Все графики сохранены в папку plots/

```

Рисунок 4.22: Запуск скрипта scripts/task_8_5_7.jl

```

In [ ]: using Random, StatsPlots, DataFrames
include("../src/sir_model.jl")

Расширенная структура для SEIR

In [ ]: mutable struct SEIRModel
    sim::ConcurrentSim.Simulation
    β::Float64
    c::Float64
    γ::Float64
    σ::Float64 # Интенсивность перехода E -> I
    ta::Array{Float64}
    Sa::Array{Int64}
    Ea::Array{Int64} # Латентные
    Ia::Array{Int64}
    Ra::Array{Int64}
    allIndividuals::Array{SIRPerson}
end

Вспомогательные функции для массивов (если не импортированы)

In [ ]: function increment!(a::Array{Int64})
    push!(a, a[end] + 1)
end

function decrement!(a::Array{Int64})
    push!(a, a[end] - 1)
end

function carryover!(a::Array{Int64})
    push!(a, a[end])
end

Обновление статистики для SEIR

In [ ]: function to_E_update!(sim::ConcurrentSim.Simulation, m::SEIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
    decrement!(m.Sa)
    increment!(m.Ea)
    carryover!(m.Ia)
end

```

Рисунок 4.23: jupyter-notebook от scripts/task_8_5_7.j

Создаем производные форматы от скриптов

```

t=. scripts/tangle.jl scripts/task_8_5_7.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_7.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_7/task_8_5_7.jl`
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_7.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/markdown/task_8_5_7/task_8_5_7.qmd`
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_7.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/notebooks/task_8_5_7/task_8_5_7.ipynb`
Готово!

```

```
t=. scripts/tangle.jl scripts/task_8_5_5.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_5.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_5/task_8_5_5.jl`
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_5.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/markdown/task_8_5_5/task_8_5_5.qmd`
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_5.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/notebooks/task_8_5_5/task_8_5_5.ipynb`
Готово!
```

```
t=. scripts/tangle.jl scripts/task_8_5_3.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_3.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_3/task_8_5_3.jl`
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_3.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/markdown/task_8_5_3/task_8_5_3.qmd`
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_3.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/notebooks/task_8_5_3/task_8_5_3.ipynb`
Готово!
```

```
t=. scripts/tangle.jl scripts/task_8_5_1.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_1.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_1/task_8_5_1.jl`
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_1.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/markdown/task_8_5_1/task_8_5_1.qmd`
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/task_8_5_1.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/notebooks/task_8_5_1/task_8_5_1.ipynb`
Готово!
```

```
ect=. scripts/tangle.jl sir_model.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/sir_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/scripts/sir_model/sir_model.jl`
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/sir_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/markdown/sir_model/sir_model.qmd`
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/sir_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab08/project/notebooks/sir_model/sir_model.ipynb`
Готово!
```

В каталоге отчёта в файл `_quarto.yml` включаем поддержку кода `julia`


```

! _quarto.yml X
Ubuntu > home > desti > work > study > 2026-1 > 2026-1==study--simulation-modeling
1  project:
2    title: "report"
3    output-dir: "_output"
4
5  standalone: true
6  self-contained: true
7
8  ## Julia support
9  # engine: julia
10 # julia:
11 #   exeflags: ["--project=../project"]
12
13 ## Generic options
14 lang: ru-RU
15 number-sections: true
16 toc: true
17 toc-title: "Содержание"
18 toc-depth: 2
19 ## Crossref customization
20 crossref:
21   lof-title: "Список иллюстраций"
22   lot-title: "Список таблиц"
23   lol-title: "Листинги"
24 ## BibliographyS
25 bibliography:
26   - bib/cite.bib
27   csl: _resources/csl/gost-r-7-0-5-2008-numeric.csl
28 ## Formats
29 format:
30   ### Docx output format
31   docx:
32     toc: true
33     number-sections: true
34     toc-depth: 2
35   ### Pdf output format
36   pdf:
37     # pdf-engine: xelatex
38     ## mnted support
39     # filters:
40     #   - mnted-quarto
41     toc: true
42     number-sections: true
43     colorlinks: false
44     toc-depth: 2
45     lof: true # List of figures
46     lot: true # List of tables
47     #### Document
48     documentclass: scrreprt
49     papersize: a4
50     fontsize: 12pt
51     linestretch: 1.5
52     #### Language
53     babel-lang: russian
54     babel-otherlangs: english
55     #### Biblatex
56     cite-method: biblatex
57     biblio-style: gost-numeric
58     biblatexoptions:
59       - backend=biber
60       - langhook=extras
61       - autolang=other*
62     #### Misc options
63     csquotes: true
64     indent: true
65     include-in-header:
66       - file: _resources/tex/preamble.tex
67

```

Рисунок 4.24: Настройка _quarto.yml

В файле отчёта подключаем файл описания программ

```
{{< include ../project/markdown/sir_des/sir_des.qmd >}}  
{{< include ../project/markdown/sir_model/sir_model.qmd >}}  
{{< include ../project/markdown/task_8_5_1/task_8_5_1.qmd >}}  
{{< include ../project/markdown/task_8_5_2/task_8_5_2.qmd >}}  
{{< include ../project/markdown/task_8_5_3/task_8_5_3.qmd >}}  
{{< include ../project/markdown/task_8_5_4/task_8_5_4.qmd >}}  
{{< include ../project/markdown/task_8_5_5/task_8_5_5.qmd >}}  
{{< include ../project/markdown/task_8_5_6/task_8_5_6.qmd >}}  
{{< include ../project/markdown/task_8_5_7/task_8_5_7.qmd >}}
```

Рисунок 4.25: Подключение файлов

```
using DrWatson  
@quickactivate "project"  
include(sourcedir("sir_model.jl"))  
using Random, StatsPlots, BenchmarkTools
```

Параметры модели

```
tmax = 40.0  
u0 = [990, 10, 0] # S, I, R  
p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$   
Random.seed!(1234)
```

Запуск модели

```
des_model = MakeSIRModel(u0, p)  
activate(des_model)  
sir_run(des_model, tmax)  
data_des = out(des_model)
```

Визуализация

```
@df data_des plot(
:t,
[:S :I :R],
labels = ["S" "I" "R"],
xlab = "Время",
ylab = "Численность",
title = "Дискретно-событийная SIR модель",
)
savefig(plotsdir("sir_des.png"))
```

```
using ResumableFunctions, ConcurrentSim, Distributions, DataFrames, Random
```

Вспомогательные функции для обновления массивов состояния

```
function increment!(a::Array{Int64})
push!(a, a[length(a)] + 1)
end
function decrement!(a::Array{Int64})
push!(a, a[length(a)] - 1)
end
function carryover!(a::Array{Int64})
push!(a, a[length(a)])
end
```

Структуры данных

```
mutable struct SIRPerson
id::Int64
status::Symbol # :S, :I, :R
end
```

```

mutable struct SIRModel
  sim::ConcurrentSim.Simulation # Тип Simulation, не Environment
  i::Float64
  c::Float64
  r::Float64
  ta::Array{Float64}
  Sa::Array{Int64}
  Ia::Array{Int64}
  Ra::Array{Int64}
  allIndividuals::Array{SIRPerson}
end

```

Функции обновления статистики при событиях

```

function infection_update!(sim::ConcurrentSim.Simulation,
m::SIRModel)
  push!(m.ta, ConcurrentSim.now(sim))
  decrement!(m.Sa)
  increment!(m.Ia)
  carryover!(m.Ra)
end

function recovery_update!(sim::ConcurrentSim.Simulation, m::SIRModel)
  push!(m.ta, ConcurrentSim.now(sim))
  carryover!(m.Sa)
  decrement!(m.Ia)
  increment!(m.Ra)
end

```

Основная логика жизни индивида

```

@resumable function live(env::ConcurrentSim.Simulation, individual::SIRPerson, m::SIRModel)
while individual.status == :S
@yield timeout(env, rand(Exponential(1/m.c)))
alter = individual
while alter == individual
N = length(m.allIndividuals)
index = rand(DiscreteUniform(1, N))
alter = m.allIndividuals[index]
end
if alter.status == :I
if rand(Uniform(0, 1)) < m.β
individual.status = :I
infection_update!(env, m)
end
end
end
if individual.status == :I
@yield timeout(env, rand(Exponential(1/m.β)))
individual.status = :R
recovery_update!(env, m)
end
end
end

```

Функции создания и запуска модели

```

function MakeSIRModel(u0, p)
(S, I, R) = u0
N = S + I + R
(β, c, γ) = p

```

```

sim = ConcurrentSim.Simulation() # Создаём именно Simulation
allIndividuals = SIRPerson[]
for i = 1:S
push!(allIndividuals, SIRPerson(i, :S))
end
for i = (S+1):(S+I)
push!(allIndividuals, SIRPerson(i, :I))
end
for i = (S+I+1):N
push!(allIndividuals, SIRPerson(i, :R))
end
ta = Float64[0.0]
Sa = Int64[S]
Ia = Int64[I]
Ra = Int64[R]
SIRModel(sim, [], c, [], ta, Sa, Ia, Ra, allIndividuals)
end
function activate(m::SIRModel)
[@process live(m.sim, individual, m) for individual in m.allIndividuals]
end
function sir_run(m::SIRModel, tf::Float64)
ConcurrentSim.run(m.sim, tf)
end
function out(m::SIRModel)
result = DataFrame()
result[:, :t] = m.ta
result[:, :S] = m.Sa
result[:, :I] = m.Ia

```

```

result[!, :R] = m.Ra
return result
end

```

```

using Random, DataFrames, CSV, StatsPlots
include("../src/sir_model.jl")

function sensitivity_analysis()
    u0 = [990, 10, 0]
    tmax = 40.0

```

Варьируемые параметры

```

    betas = [0.03, 0.05, 0.07]
    cs = [5.0, 10.0, 15.0]
    gammas = [0.2, 0.25, 0.3]

    results = DataFrame(
        i=Float64[], c=Float64[], r=Float64[],
        peak_I=Int64[], peak_time=Float64[], final_R=Int64[],
        total_infected=Int64[]
    )

    for β in betas
        for c in cs
            for γ in gammas
                println("Запуск: β=$β, c=$c, γ=$γ")

                Random.seed!(1234) # Фиксируем seed для воспроизводимости

```

```

        p = [u0, c, u0]
        m = MakeSIRModel(u0, p)
        activate(m)
        sir_run(m, tmax)
        data = out(m)

        peak_I = maximum(data.I)
        peak_time = data.t[argmax(data.I)]
        final_R = last(data.R)
        total_infected = final_R - u0[3] # Сколько заболело всего

        push!(results, (c, c, c, peak_I, peak_time, final_R, total_infected))
    end
end
end
end

```

Сохраняем результаты

```

CSV.write("data/sensitivity_analysis.csv", results)
println("\n=== РЕЗУЛЬТАТЫ АНАЛИЗА ЧУВСТВИТЕЛЬНОСТИ ===")
println(results)

```

Визуализация

```

for c in unique(results.c)
    filtered = filter(row -> row.c == c, results)
    p1 = plot(filtered.t, filtered.peak_I, group=filtered.c,
              xlabel="t", ylabel="Пик I", title="c=$c",
              label=["c=5" "c=10" "c=15"], marker=:circle)
    savefig(p1, "plots/sensitivity_$(c).png")
end

```



```

        end

        return results
    end

    sensitivity_analysis()

```

```

using Random, StatsPlots, DataFrames
include("../src/sir_model.jl")

```

Модифицированная версия live с фиксированным временем болезни

```

@resumable function live_deterministic(env::ConcurrentSim.Simulation,
                                     individual::SIRPerson,
                                     m::SIRModel)

    while individual.status == :S
        @yield timeout(env, rand(Exponential(1/m.c)))
    end

```

Выбор случайного собеседника

```

        alter = individual
        while alter == individual
            N = length(m.allIndividuals)
            index = rand(DiscreteUniform(1, N))
            alter = m.allIndividuals[index]
        end

        if alter.status == :I
            if rand() < m.β
                individual.status = :I
            end
        end
    end

```

```

        infection_update!(env, m)
    end
end
end

if individual.status == :I

```

ФИКСИРОВАННОЕ время болезни (вместо экспоненциального)

```

        @yield timeout(env, 1/m.□)
        individual.status = :R
        recovery_update!(env, m)
    end
end

```

Переопределяем функцию activate для использования детерминированной версии

```

function activate_deterministic(m::SIRModel)
    for individual in m.allIndividuals
        @process live_deterministic(m.sim, individual, m)
    end
end

```

Сравнение двух версий

```

function compare_stochastic_deterministic()
    u0 = [990, 10, 0]
    p = [0.05, 10.0, 0.25]
    tmax = 40.0

```

Стохастическая версия

```

Random.seed!(1234)
m_stoch = MakeSIRModel(u0, p)
activate(m_stoch)
sir_run(m_stoch, tmax)
data_stoch = out(m_stoch)

```

Детерминированная версия

```

Random.seed!(1234)
m_det = MakeSIRModel(u0, p)
activate_deterministic(m_det)
sir_run(m_det, tmax)
data_det = out(m_det)

```

График сравнения

```

plot(data_stoch.t, [data_stoch.S data_stoch.I data_stoch.R],
     label=["S (стох)" "I (стох)" "R (стох)"],
     linestyle=:solid, linewidth=2, xlabel="Время", ylabel="Численность",
     title="Сравнение SIR: стохастическая vs детерминированная")

plot!(data_det.t, [data_det.S data_det.I data_det.R],
     label=["S (дет)" "I (дет)" "R (дет)"],
     linestyle=:dash, linewidth=2)

savefig("plots/comparison_stoch_det.png")
println("График сохранён в plots/comparison_stoch_det.png")

return (stochastic=data_stoch, deterministic=data_det)
end

```

```
compare_stochastic_deterministic()
```

```
using BenchmarkTools, Random
include("../src/sir_model.jl")

function performance_test()
    println("=== ОЦЕНКА ПРОИЗВОДИТЕЛЬНОСТИ ===\n")
```

Различные размеры популяции

```
population_sizes = [100, 500, 1000, 2000, 5000]
results = DataFrame(N=Int[], time_ms=Float64[], memory_MB=Float64[])

for N in population_sizes
    S = N - 10
    I = 10
    R = 0
    u0 = [S, I, R]
    p = [0.05, 10.0, 0.25]
    tmax = 40.0

    m = MakeSIRModel(u0, p)
    activate(m)
```

Бенчмарк

```
bench = @benchmark sir_run($m, $tmax) samples=10

time_ms = BenchmarkTools.minimum(bench).time / 1e6 # в миллисекундах
```

```

        memory_MB = BenchmarkTools.minimum(bench).memory / 1024 / 1024

        push!(results, (N, time_ms, memory_MB))
        println("N=$N: время = $(round(time_ms, digits=2)) мс, память = $(round(memory_MB, digits=2)) МБ")
    end
end

```

Визуализация

```

plot(results.N, results.time_ms, marker=:circle, linewidth=2,
      xlabel="Размер популяции (N)", ylabel="Время выполнения (мс)",
      title="Производительность SIR модели", legend=false)
savefig("plots/performance.png")

println("\n=== СОВЕТЫ ПО ОПТИМИЗАЦИИ ===")
println("1. Использовать предварительное выделение массивов")
println("2. Генерировать случайные числа блоками (вместо поштучно)")
println("3. Использовать Threads.@threads для параллельных прогонов")
println("4. Хранить статусы в массиве Int8 вместо Symbol")

return results
end

performance_test()

```

```

using Random, DataFrames, CSV, Dates
include("../src/sir_model.jl")

function save_results_with_metadata(data::DataFrame, u0, p, tmax)

```

Создаём директорию, если её нет

```
mkpath("data/sims")
```

Формируем уникальное имя файла

```
timestamp = Dates.format(now(), "YYYYmmdd_HHMMSS")
filename = "sir_S=$(u0[1])_I=$(u0[2])_R=$(u0[3])_□=$(p[1])_c=$(p[2])_□=$(p[3])_tm
filepath = joinpath("data", "sims", filename)
```

Сохраняем данные

```
CSV.write(filepath, data)
```

Сохраняем метаданные в отдельный файл

```
metadata = DataFrame(
    parameter=["S0", "I0", "R0", "□", "c", "□", "tmax", "timestamp", "seed"],
    value=[u0[1], u0[2], u0[3], p[1], p[2], p[3], tmax, timestamp, 1234]
)
metadata_path = replace(filepath, ".csv" => "_metadata.csv")
CSV.write(metadata_path, metadata)

println("□ Результаты сохранены:")
println("    Данные: $filepath")
println("    Метаданные: $metadata_path")

return filepath
end

function run_and_save()
```

Параметры модели

```
tmax = 40.0
u0 = [990, 10, 0]
p = [0.05, 10.0, 0.25]

Random.seed!(1234)
```

Запуск модели

```
m = MakeSIRModel(u0, p)
activate(m)
sir_run(m, tmax)
data = out(m)
```

Сохранение

```
saved_path = save_results_with_metadata(data, u0, p, tmax)
```

Проверка: читаем сохранённый файл

```
data_loaded = CSV.read(saved_path, DataFrame)
println("\nПервые 5 строк сохранённых данных:")
println(first(data_loaded, 5))

return data_loaded
end

run_and_save()
```

```
using Random, StatsPlots, DataFrames, ConcurrentSim, ResumableFunctions, Distribution
include("../src/sir_model.jl")
```

Расширенная структура с демографией

```
mutable struct SIRModelDemography
    sim::ConcurrentSim.Simulation
    i::Float64
    c::Float64
    r::Float64
    d::Float64      # Интенсивность смертности
    b::Float64      # Интенсивность рождаемости
    ta::Array{Float64}
    Sa::Array{Int64}
    Ia::Array{Int64}
    Ra::Array{Int64}
    Da::Array{Int64}    # Смерти
    allIndividuals::Array{SIRPerson}
end
```

Вспомогательные функции для массивов (если не импортированы из sir_model.jl)

```
function increment!(a::Array{Int64})
    push!(a, a[end] + 1)
end

function decrement!(a::Array{Int64})
    push!(a, a[end] - 1)
end

function carryover!(a::Array{Int64})
    push!(a, a[end])
end
```



```
end
```

Обновление статистики при заражении (из базовой модели)

```
function infection_update!(sim::ConcurrentSim.Simulation, m::SIRModelDemography)
    push!(m.ta, ConcurrentSim.now(sim))
    decrement!(m.Sa)
    increment!(m.Ia)
    carryover!(m.Ra)
    carryover!(m.Da)
end
```

Обновление статистики при выздоровлении

```
function recovery_update!(sim::ConcurrentSim.Simulation, m::SIRModelDemography)
    push!(m.ta, ConcurrentSim.now(sim))
    carryover!(m.Sa)
    decrement!(m.Ia)
    increment!(m.Ra)
    carryover!(m.Da)
end
```

Обновление статистики при смерти

```
function death_update!(sim::ConcurrentSim.Simulation, m::SIRModelDemography, individual)
    push!(m.ta, ConcurrentSim.now(sim))
    if individual.status == :S
        decrement!(m.Sa)
        carryover!(m.Ia)
        carryover!(m.Ra)
    elseif individual.status == :I
```

```

        carryover!(m.Sa)
        decrement!(m.Ia)
        carryover!(m.Ra)
    elseif individual.status == :R
        carryover!(m.Sa)
        carryover!(m.Ia)
        decrement!(m.Ra)
    end
    increment!(m.Da)
end

```

Обновление статистики при рождении

```

function birth_update!(sim::ConcurrentSim.Simulation, m::SIRModelDemography)
    push!(m.ta, ConcurrentSim.now(sim))
    increment!(m.Sa)
    carryover!(m.Ia)
    carryover!(m.Ra)
    carryover!(m.Da)
end

```

Жизненный цикл с демографией (объединённый процесс)

```

@resumable function live_with_demography(env::ConcurrentSim.Simulation,
                                          individual::SIRPerson,
                                          m::SIRModelDemography)

```

Переменная для отслеживания, жив ли индивид

```
alive = true
```

```
while alive
```

Если индивид восприимчив

```
while individual.status == :S && alive
```

Ожидание до следующего события (контакт или смерть)

```
contact_time = rand(Exponential(1/m.c))
```

```
death_time = rand(Exponential(1/m.□))
```

```
if contact_time < death_time
```

Контакт произошёл раньше смерти

```
@yield timeout(env, contact_time)
```

```
if !alive
```

```
break
```

```
end
```

Выбор случайного собеседника

```
alter = individual
```

```
while alter == individual
```

```
  N = length(m.allIndividuals)
```

```
  index = rand(DiscreteUniform(1, N))
```

```
  alter = m.allIndividuals[index]
```

```
end
```

Заражение

```
        if alter.status == :I
          if rand() < m.β
            individual.status = :I
            infection_update!(env, m)
            break # Выходим из цикла восприимчивых
          end
        end
      end
    else
```

Смерть произошла раньше контакта

```
      @yield timeout(env, death_time)
      if individual.status != :D
        death_update!(env, m, individual)
        individual.status = :D
        alive = false
      end
      break
    end
  end
end
```

Если индивид инфицирован

```
    while individual.status == :I && alive
```

Ожидание до следующего события (выздоровление или смерть)

```

    recovery_time = rand(Exponential(1/m.□))
    death_time = rand(Exponential(1/m.□))

    if recovery_time < death_time
        @yield timeout(env, recovery_time)
        if individual.status == :I && alive
            individual.status = :R
            recovery_update!(env, m)
        end
        break
    else
        @yield timeout(env, death_time)
        if individual.status != :D && alive
            death_update!(env, m, individual)
            individual.status = :D
            alive = false
        end
        break
    end
end
end

```

Если индивид переболел

```
while individual.status == :R && alive
```

Ожидание смерти

```

    @yield timeout(env, rand(Exponential(1/m.□)))
    if individual.status != :D
        death_update!(env, m, individual)
    end
end

```

```

        individual.status = :D
        alive = false
    end
    break
end
end
end
end

```

Процесс рождения

```

@resumable function birth_process(env::ConcurrentSim.Simulation, m::SIRModelDemograph
    while true
        @yield timeout(env, rand(Exponential(1/m.□)))
    end
end

```

Создаём нового индивида

```

new_id = length(m.allIndividuals) + 1
new_person = SIRPerson(new_id, :S)
push!(m.allIndividuals, new_person)
birth_update!(env, m)

```

Запускаем процесс для новорождённого

```

    @process live_with_demography(env, new_person, m)
end
end

```

Создание модели с демографией

```

function MakeSIRModelDemography(u0, p, S, I, R)
    (S, I, R) = u0
    N = S + I + R
    (c, c, c) = p

    sim = ConcurrentSim.Simulation()
    allIndividuals = SIRPerson[]

```

Создаём индивидов с начальными статусами

```

    id = 1
    for i in 1:S
        push!(allIndividuals, SIRPerson(id, :S))
        id += 1
    end
    for i in 1:I
        push!(allIndividuals, SIRPerson(id, :I))
        id += 1
    end
    for i in 1:R
        push!(allIndividuals, SIRPerson(id, :R))
        id += 1
    end

    ta = Float64[0.0]
    Sa = Int64[S]
    Ia = Int64[I]
    Ra = Int64[R]
    Da = Int64[0]

```

```

        return SIRModelDemography(sim, [], c, [], [], [], ta, Sa, Ia, Ra, Da, allIndividuals)
    end

```

Активация всех процессов

```

function activate_demography(m::SIRModelDemography)
    for individual in m.allIndividuals
        @process live_with_demography(m.sim, individual, m)
    end
    @process birth_process(m.sim, m)
end

```

Запуск симуляции

```

function sir_run_demography(m::SIRModelDemography, tf::Float64)
    ConcurrentSim.run(m.sim, tf)
end

```

Сбор результатов

```

function out_demography(m::SIRModelDemography)
    result = DataFrame()
    result[:, :t] = m.ta
    result[:, :S] = m.Sa
    result[:, :I] = m.Ia
    result[:, :R] = m.Ra
    result[:, :D] = m.Da
    return result
end

```

Основная функция запуска


```

function run_demography()
  u0 = [990, 10, 0]
  p = [0.05, 10.0, 0.25]
   $\mu$  = 0.01 # интенсивность смерти (средняя продолжительность жизни = 100 ед. времени)
   $\lambda$  = 0.012 # интенсивность рождения (чуть выше, чтобы популяция росла)
  tmax = 100.0

  println("=== МОДЕЛЬ SIR С ДЕМОГРАФИЕЙ ===")
  println("Параметры:")
  println(" - Смертность ( $\mu$ ):  $\mu$  (средняя жизнь =  $1/\mu$  ед. времени)")
  println(" - Рождаемость ( $\lambda$ ):  $\lambda$  (средний интервал =  $1/\lambda$  ед. времени)")
  println(" - Длительность симуляции:  $t_{max}$ \n")

  Random.seed!(1234)
  m = MakeSIRModelDemography(u0, p,  $\mu$ ,  $\lambda$ )
  activate_demography(m)
  sir_run_demography(m, tmax)
  data = out_demography(m)

```

Сохраняем данные в CSV

```

mkpath("data")
CSV.write("data/sir_demography.csv", data)
println("\n Данные сохранены в data/sir_demography.csv")

```

Визуализация

```

p1 = plot(data.t, [data.S data.I data.R],
          label=["S (Восприимчивые)" "I (Инфицированные)" "R (Переболевшие)"],
          linewidth=2,

```

```

        xlabel="Время", ylabel="Численность",
        title="SIR модель с демографическими событиями")

p2 = plot(data.t, data.D, label="Смерти", linewidth=2, color=:purple,
        xlabel="Время", ylabel="Численность", title="Накопленные смерти")

```

Общая численность популяции

```

total_pop = data.S .+ data.I .+ data.R
p3 = plot(data.t, total_pop, label="Общая популяция", linewidth=2, color=:green,
        xlabel="Время", ylabel="Численность", title="Динамика популяции")

p_all = plot(p1, p2, p3, layout=(3,1), size=(800, 900))
mkpath("plots")
savefig(p_all, "plots/sir_demography.png")

println("\n=== ИТОГОВЫЕ ПОКАЗАТЕЛИ ===")
println("Начальная популяция: $(u0[1]+u0[2]+u0[3])")
println("Конечная популяция: $(last(data.S) + last(data.I) + last(data.R))")
println("Всего умерло: $(last(data.D))")
println("Всего переболело: $(last(data.R))")
println("Активно инфицированы в конце: $(last(data.I))")

```

Расчёт эндемического уровня

```

if tmax > 50
    endemic_I = mean(data.I[end-20:end])
    println("\nЭндемический уровень инфекции: $(round(endemic_I, digits=2))")
end

```

```
println("\n□ Графики сохранены в plots/sir_demography.png")

return data
end
```

Запуск

```
run_demography()
```

```
using Random, StatsPlots, DataFrames
include("../src/sir_model.jl")
```

Функция вакцинации

```
@resumable function vaccinate(env::ConcurrentSim.Simulation, m::SIRModel,
                              vaccine_time::Float64, fraction::Float64)
    @yield timeout(env, vaccine_time)
```

Собираем всех восприимчивых

```
susceptible = [ind for ind in m.allIndividuals if ind.status == :S]
n_to_vaccinate = round{Int}(fraction * length(susceptible))

if n_to_vaccinate > 0
```

Выбираем случайных восприимчивых для вакцинации

```
to_vaccinate = sample(susceptible, n_to_vaccinate, replace=false)

for ind in to_vaccinate
    ind.status = :R
end
```

Обновляем статистику

```
push!(m.ta, now(env))
m.Sa[end] -= n_to_vaccinate
m.Ra[end] += n_to_vaccinate

println("□ Вакцинировано $n_to_vaccinate человек в момент времени $(round(now(), :date{yyyy-mm-dd}))")
else
println("□ Нет восприимчивых для вакцинации")
end
end
end
```

Функция для исследования эффективности вакцинации

```
function vaccination_study()
    u0 = [990, 10, 0]
    p = [0.05, 10.0, 0.25]
    tmax = 60.0
    vaccine_time = 10.0 # вакцинация на 10-й день

    fractions = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]
    results = DataFrame(fraction=Float64[], peak_I=Int64[], final_R=Int64[], epidemic=Bool[])

    for fraction in fractions
        println("Запуск с долей вакцинации: $(fraction*100)%")

        Random.seed!(1234)
        m = MakeSIRModel(u0, p)
        activate(m)
```

```

    if fraction > 0
        @process vaccinate(m.sim, m, vaccine_time, fraction)
    end

    sir_run(m, tmax)
    data = out(m)

    peak_I = maximum(data.I)
    final_R = last(data.R)
    epidemic_occurred = peak_I > 50 ? "Да" : "Нет"

    push!(results, (fraction, peak_I, final_R, epidemic_occurred))

```

Сохраняем график для каждой доли

```

    plot(data.t, [data.S data.I data.R],
        label=["S" "I" "R"], linewidth=2,
        xlabel="Время", ylabel="Численность",
        title="Вакцинация $(fraction*100)% в момент t=$vaccine_time")
    vline!([vaccine_time], label="Вакцинация", linestyle=:dash, color=:black)
    savefig("plots/vaccination_$(fraction*100)percent.png")
end

```

Результаты

```

println("\n=== РЕЗУЛЬТАТЫ ВАКЦИНАЦИИ ===")
println(results)

```

График эффективности

```

plot(results.fraction * 100, results.peak_I, marker=:circle, linewidth=2,
      xlabel="Доля вакцинированных (%)", ylabel="Пик эпидемии (max I)",
      title="Эффективность вакцинации", legend=false)
savefig("plots/vaccination_efficiency.png")

```

Критическая доля для предотвращения эпидемии

```

critical_fraction = minimum(results.fraction[results.epidemic .== "Нет"])
println("\n□ Критическая доля вакцинации для предотвращения эпидемии: $(critical_

return results
end

vaccination_study()

```

```

using Random, StatsPlots, DataFrames
include("../src/sir_model.jl")

```

Расширенная структура для SEIR

```

mutable struct SEIRModel
    sim::ConcurrentSim.Simulation
    β::Float64
    c::Float64
    γ::Float64
    βI::Float64      # Интенсивность перехода E -> I
    ta::Array{Float64}
    Sa::Array{Int64}
    Ea::Array{Int64}  # Латентные
    Ia::Array{Int64}

```

```

    Ra::Array{Int64}
    allIndividuals::Array{SIRPerson}
end

```

Вспомогательные функции для массивов (если не импортированы)

```

function increment!(a::Array{Int64})
    push!(a, a[end] + 1)
end

function decrement!(a::Array{Int64})
    push!(a, a[end] - 1)
end

function carryover!(a::Array{Int64})
    push!(a, a[end])
end

```

Обновление статистики для SEIR

```

function to_E_update!(sim::ConcurrentSim.Simulation, m::SEIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
    decrement!(m.Sa)
    increment!(m.Ea)
    carryover!(m.Ia)
    carryover!(m.Ra)
end

function to_I_update!(sim::ConcurrentSim.Simulation, m::SEIRModel)
    push!(m.ta, ConcurrentSim.now(sim))

```

```

    carryover!(m.Sa)
    decrement!(m.Ea)
    increment!(m.Ia)
    carryover!(m.Ra)
end

function recovery_update_SEIR!(sim::ConcurrentSim.Simulation, m::SEIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
    carryover!(m.Sa)
    carryover!(m.Ea)
    decrement!(m.Ia)
    increment!(m.Ra)
end

```

Жизненный цикл для SEIR

```

@resumable function live_SEIR(env::ConcurrentSim.Simulation,
                             individual::SIRPerson,
                             m::SEIRModel)

    while individual.status == :S
        @yield timeout(env, rand(Exponential(1/m.c)))

        alter = individual
        while alter == individual
            N = length(m.allIndividuals)
            index = rand(DiscreteUniform(1, N))
            alter = m.allIndividuals[index]
        end
    end
end

```



```

    if alter.status == :I
        if rand() < m.□
            individual.status = :E # Латентный период
            to_E_update!(env, m)
        end
    end
end

```

Планируем переход E -> I

```

        @yield timeout(env, rand(Exponential(1/m.□)))
        if individual.status == :E
            individual.status = :I
            to_I_update!(env, m)
        end
    end
end

```

Выздоровление

```

        @yield timeout(env, rand(Exponential(1/m.□)))
        if individual.status == :I
            individual.status = :R
            recovery_update_SEIR!(env, m)
        end
    end
end
break
end
end
end
end

```

Создание SEIR модели

```

function MakeSEIRModel(u0, p, □)
    (S, E, I, R) = u0
    m = SEIRModel(p, □)
    env = Dict{String, Any}()
    env["m"] = m
    env["individuals"] = Dict{Int, SEIRIndividual}()
    for i in 1:length(u0)
        env["individuals"][i] = SEIRIndividual(i, m, u0[i])
    end
    return env
end

```

```
N = S + E + I + R
```

```
(□, c, □) = p
```

```
sim = ConcurrentSim.Simulation()
```

```
allIndividuals = SIRPerson[]
```

Создаём индивидов с начальными статусами

```
id = 1
```

```
for i in 1:S
```

```
    push!(allIndividuals, SIRPerson(id, :S))
```

```
    id += 1
```

```
end
```

```
for i in 1:E
```

```
    push!(allIndividuals, SIRPerson(id, :E))
```

```
    id += 1
```

```
end
```

```
for i in 1:I
```

```
    push!(allIndividuals, SIRPerson(id, :I))
```

```
    id += 1
```

```
end
```

```
for i in 1:R
```

```
    push!(allIndividuals, SIRPerson(id, :R))
```

```
    id += 1
```

```
end
```

```
ta = Float64[0.0]
```

```
Sa = Int64[S]
```

```
Ea = Int64[E]
```

```

    Ia = Int64[I]
    Ra = Int64[R]

    return SEIRModel(sim, [], c, [], [], ta, Sa, Ea, Ia, Ra, allIndividuals)
end

function activate_SEIR(m::SEIRModel)
    for individual in m.allIndividuals
        @process live_SEIR(m.sim, individual, m)
    end
end
end

```

Функция запуска SEIR модели (аналог sir_run)

```

function seir_run(m::SEIRModel, tf::Float64)
    ConcurrentSim.run(m.sim, tf)
end

```

Функция сбора результатов для SEIR

```

function out_SEIR(m::SEIRModel)
    result = DataFrame()
    result[:, :t] = m.ta
    result[:, :S] = m.Sa
    result[:, :E] = m.Ea
    result[:, :I] = m.Ia
    result[:, :R] = m.Ra
    return result
end

```

Сравнение SIR vs SEIR

```

function compare_SIR_SEIR()
    u0_SIR = [990, 10, 0]
    u0_SEIR = [990, 0, 10, 0] # S, E, I, R
    p = [0.05, 10.0, 0.25]
     $\tau$  = 0.5 # Средняя длительность латентного периода = 2 дня (1/ $\tau$ )
    tmax = 60.0

    println("=== Запуск SIR модели ===")
    Random.seed!(1234)
    m_sir = MakeSIRModel(u0_SIR, p)
    activate(m_sir)
    sir_run(m_sir, tmax)
    data_sir = out(m_sir)

    println("=== Запуск SEIR модели ===")
    Random.seed!(1234)
    m_seir = MakeSEIRModel(u0_SEIR, p,  $\tau$ )
    activate_SEIR(m_seir)
    seir_run(m_seir, tmax)
    data_seir = out_SEIR(m_seir)

```

Сравнительные графики

```

p1 = plot(data_sir.t, data_sir.I,
          label="I (SIR)", linewidth=2, linestyle=:solid,
          xlabel="Время", ylabel="Численность инфицированных",
          title="SIR vs SEIR: сравнение динамики инфекции")

plot!(data_seir.t, data_seir.I,

```

```

        label="I (SEIR)", linewidth=2, linestyle=:dash)

savefig(p1, "plots/SIR_vs_SEIR_I.png")

```

График всех состояний SEIR

```

p2 = plot(data_seir.t, [data_seir.S data_seir.E data_seir.I data_seir.R],
        label=["S" "E" "I" "R"], linewidth=2,
        xlabel="Время", ylabel="Численность",
        title="SEIR модель: все состояния")

savefig(p2, "plots/SEIR_all_states.png")

```

Статистика

```

println("\n=== РЕЗУЛЬТАТЫ СРАВНЕНИЯ ===")
println("SIR модель:")
println(" - Пик инфекции: $(maximum(data_sir.I)) в момент t = $(data_sir.t[argmax(data_sir.I)])")
println(" - Всего переболело: $(last(data_sir.R))")

println("\nSEIR модель (с латентным периодом 1/□ = $(1/□) дней):")
println(" - Пик инфекции (I): $(maximum(data_seir.I)) в момент t = $(data_seir.t[argmax(data_seir.I)])")
println(" - Пик латентных (E): $(maximum(data_seir.E)) в момент t = $(data_seir.t[argmax(data_seir.E)])")
println(" - Всего переболело: $(last(data_seir.R))")

```

Анализ задержки

```

sir_peak_time = data_sir.t[argmax(data_sir.I)]
seir_peak_time = data_seir.t[argmax(data_seir.I)]

```

```

println("\n=== АНАЛИЗ ЗАДЕРЖКИ ===")
println("Пик инфекции в SIR: t = $(round(sir_peak_time, digits=2))")
println("Пик инфекции в SEIR: t = $(round(seir_peak_time, digits=2))")
println("Задержка пика из-за латентного периода: $(round(seir_peak_time - sir_peak_time, digits=2))")

return (sir=data_sir, seir=data_seir)
end

```

Запуск сравнения

```
result = compare_SIR_SEIR()
```

Дополнительно: исследование влияния длительности латентного периода

```

function study_latent_period()
    println("\n=== ИССЛЕДОВАНИЕ ВЛИЯНИЯ ЛАТЕНТНОГО ПЕРИОДА ===")

    u0_SEIR = [990, 0, 10, 0]
    p = [0.05, 10.0, 0.25]
    sigmas = [0.2, 0.5, 1.0, 2.0] # разные интенсивности перехода
    tmax = 60.0

    results = DataFrame{Float64[], latent_days=Float64[], peak_I=Int64[], peak_time=Float64[]}()

    for sigma in sigmas
        println("Запуск с sigma = $sigma (латентный период = $(1/sigma) дней)")

        Random.seed!(1234)
        m = MakeSEIRModel(u0_SEIR, p, sigma)
        activate_SEIR(m)
    end
end

```

```

    seir_run(m, tmax)
    data = out_SEIR(m)

    peak_I = maximum(data.I)
    peak_time = data.t[argmax(data.I)]

    push!(results, ( $\beta$ , 1/ $\beta$ , peak_I, peak_time))
end

```

Визуализация

```

p3 = plot(results.latent_days, results.peak_I, marker=:circle, linewidth=2,
          xlabel="Длительность латентного периода (дни)",
          ylabel="Пик инфекции (max I)",
          title="Влияние латентного периода на пик эпидемии", legend=false)
savefig(p3, "plots/latent_period_effect.png")

println("\nРезультаты исследования:")
println(results)

return results
end

study_latent_period()

println("\n $\square$  Все графики сохранены в папку plots/")

```

В ходе лабораторной работы:

Изучен дискретно-событийный подход к имитационному моделированию.

Реализована стохастическая SIR модель на языке Julia с использованием библиотек ConcurrentSim и ResumableFunctions.

Проведён анализ чувствительности модели к параметрам.

Выполнено сравнение стохастической и детерминированной версий.

Проведена оценка производительности для различных размеров популяции.

Реализованы расширения модели: демографические события, вакцинация, SEIR.

Полученные результаты демонстрируют, что дискретно-событийный подход позволяет гибко моделировать сложные эпидемиологические процессы и исследовать влияние различных факторов на динамику распространения инфекции.

Список литературы

Имитационное моделирование. Практикум [https://esystem.rudn.ru/pluginfile.php/3094278/
lab.pdf](https://esystem.rudn.ru/pluginfile.php/3094278/lab.pdf)